Data cleaning and quality checks

DATA TRANSFORMATION WITH SPARK SQL IN DATABRICKS



Disha Mukherjee
Lead Data Engineer

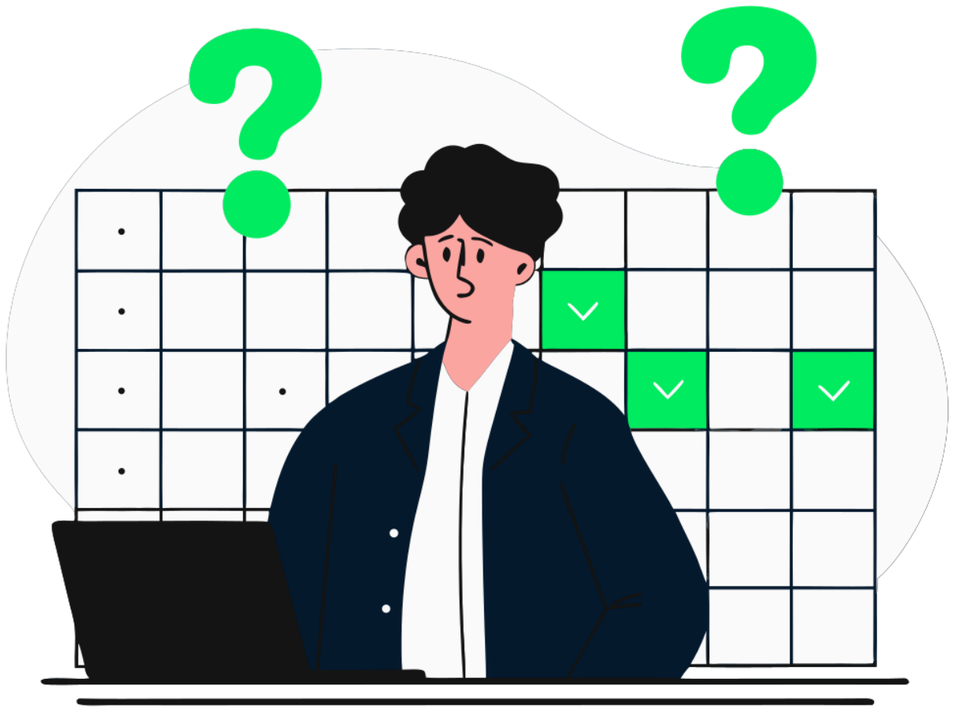
datacamp

Three problems in real data



- **Missing values** in key columns
- **Duplicate rows** from repeated ingestion
- **Logically invalid** records

Why define a schema explicitly?



- Spark can misread column types
- Numeric columns can be inferred as *strings*
- `StructType` defines types explicitly, every run

Imports

```
from pyspark.sql.types import (
    StructType, StructField,
    StringType, DoubleType, IntegerType, TimestampType
)

from pyspark.sql import functions as F
```

Defining the schema

```
schema = StructType([
    StructField("ID",                IntegerType(),   True),
    StructField("Date",              TimestampType(), True),
    StructField("Transaction_Amount", DoubleType(),   True),
    StructField("Transaction_Status", StringType(),   True),
    # ... 6 more fields
])

df = (spark.read.format("csv")
      .option("header", "true")
      .schema(schema)
      .load("/Volumes/.../transactions.csv"))

df.printSchema()
```

```
 |-- ID: integer (nullable = true)
 |-- Date: timestamp (nullable = true)
 |-- Transaction_Amount: double (nullable = true)
 |-- Transaction_Status: string (nullable = true)
 ...
```

Finding the nulls

```
null_counts = df.select([
    F.sum(F.col(c).isNull().cast("int")).alias(c)
    for c in df.columns
])

null_counts.show()
```

```
+-----------+------------------+--------+--------+
|Customer_ID|Transaction_Amount|Category|Location|
+-----------+------------------+--------+--------+
|30         |50                |40      |30      |
+-----------+------------------+--------+--------+
```

Dropping and filling nulls

```
df_no_nulls = df.na.drop(subset=["Customer_ID"])

df_filled = df_no_nulls.na.fill({
    "Transaction_Amount": 0.0,
    "Category": "Unknown",
    "Location": "Unknown"
})

print(f"Before: {df.count()}")
print(f"After:  {df_filled.count()}")
```

```
Before: 100,150
After:  100,120
```

Handling duplicates

```
total = df_filled.count()
distinct = df_filled.distinct().count()


print(f"Duplicates: {total - distinct}")
```

```
Duplicates: 149
```

Handling duplicates

```
df_deduped = df_filled.dropDuplicates(
    subset=["Customer_ID", "Date", "Transaction_Amount"]
)

print(f"Rows after dedup: {df_deduped.count()}")
```

```
Rows after dedup: 99,971
```

Filtering invalid records

```
df_valid = df_deduped.filter(F.col("Transaction_Amount") > 0)


df_valid = df_valid.filter(
    F.col("Transaction_Status") == "Completed"
)


print(f"Valid transactions: {df_valid.count()}")
```

```
Valid transactions: 33,223
```

Creating derived columns

```
df_enriched = df_valid.withColumn("Revenue_Band",
    F.when(F.col("Transaction_Amount") >= 50000, "High")
     .when(F.col("Transaction_Amount") >= 10000, "Medium")
     .otherwise("Low"))

df_enriched.select("Customer_ID", "Transaction_Amount", "Revenue_Band").show(3)
```

```
+-----------+------------------+------------+
|Customer_ID|Transaction_Amount|Revenue_Band|
+-----------+------------------+------------+
|CUST003    |5752.36           |Low         |
|CUST009    |28959.12          |Medium      |
|CUST010    |72098.18          |High        |
+-----------+------------------+------------+
```

Data quality check

```
total_rows = df_enriched.count()
distinct_rows = df_enriched.distinct().count()

null_rate = (
    df_enriched.filter(F.col("Customer_ID").isNull())
    .count() / total_rows * 100
)

print(f"Total rows:   {total_rows}")
print(f"Duplicates:   {total_rows - distinct_rows}")
print(f"Null rate:    {null_rate:.2f}%")
```

```
Total rows:   33,223
Duplicates:   0
Null rate:    0.00%
```

Let's practice!

DATA TRANSFORMATION WITH SPARK SQL IN DATABRICKS

Aggregating and joining data efficiently

DATA TRANSFORMATION WITH SPARK SQL IN DATABRICKS



Disha Mukherjee
Lead Data Engineer

datacamp

Three questions, one clean dataset



- Which **categories** drive the most revenue?
- Who are the **top customers**?
- How do we **enrich** results with department context?

How groupBy() and agg() work

- `groupBy()` - partitions rows by key column
- `agg()` - applies functions to each group in parallel
- Both are **lazy** - action triggers computation
- One `agg()` call = one data scan



Category revenue - code

```
category_revenue = (
    df_valid
    .groupBy("Category")
    .agg(
        F.round(F.sum("Transaction_Amount"), 2).alias("total_revenue"),
        F.count("Transaction_Amount").alias("transaction_count"),
        F.round(F.avg("Transaction_Amount"), 2).alias("avg_transaction"),
    )
    .orderBy(F.col("total_revenue").desc())
)
```

Category revenue - output

```
category_revenue.show(truncate=False)
```

```
+-----------+-----------------+-----------------+---------------+
|Category   |total_revenue    |transaction_count|avg_transaction|
+-----------+-----------------+-----------------+---------------+
|Clothing   |338266605.24     |6742             |50173.04       |
|Dining     |337631718.64     |6719             |50250.29       |
|Electronics|331102843.72     |6614             |50060.91       |
|Savings    |329346609.07     |6596             |49931.26       |
|Groceries  |329003451.57     |6537             |50329.43       |
|Unknown    |880591.27        |15               |58706.08       |
+-----------+-----------------+-----------------+---------------+
```

Enriching with a dimension table



```
+-----------+----------+
|Category   |Department|
+-----------+----------+
|Clothing   |Retail    |
|Dining     |Food      |
|Electronics|Tech      |
|Groceries  |Food      |
|Savings    |Finance   |
+-----------+----------+
```

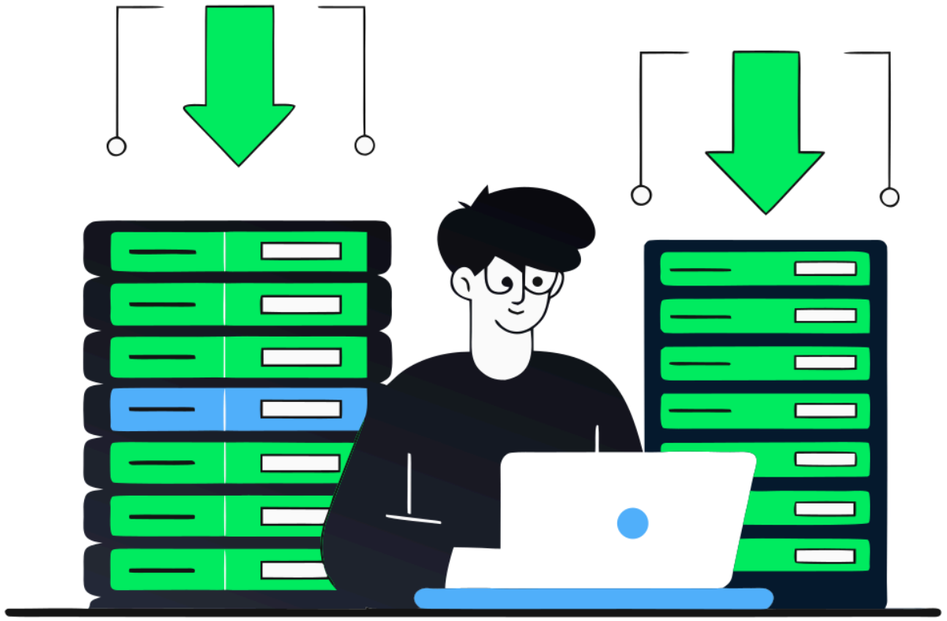
Standard left join

```
df_joined = df_valid.join(df_dim, on="Category", how="left")

df_joined.select(
    "Customer_ID", "Category", "Department", "Transaction_Amount"
).show(5, truncate=False)
```

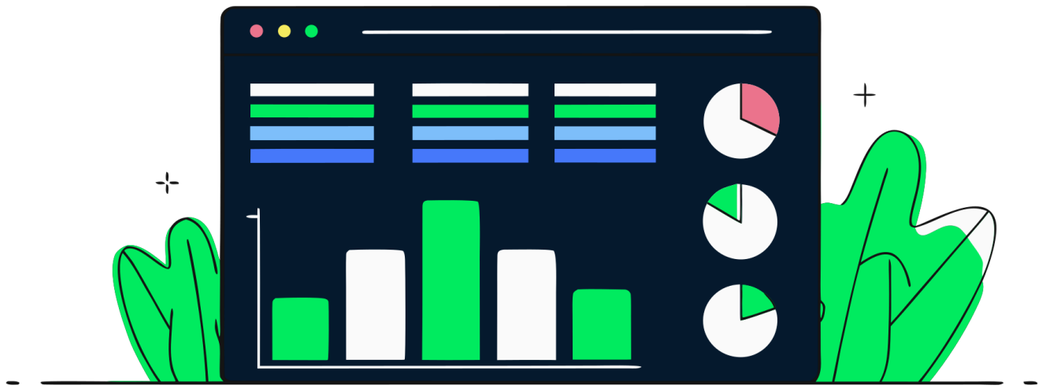
```
+-----------+-----------+----------+------------------+
|Customer_ID|Category   |Department|Transaction_Amount|
+-----------+-----------+----------+------------------+
|CUST003    |Electronics|Tech      |5752.36           |
|CUST009    |Clothing   |Retail    |28959.12          |
|CUST010    |Savings    |Finance   |72098.18          |
|CUST011    |Savings    |Finance   |49771.05          |
|CUST020    |Groceries  |Food      |69825.14          |
+-----------+-----------+----------+------------------+
```

The hidden cost - shuffle



- Matching keys must land on the **same machine**
- Spark ships rows across the network to align them
- **Shuffle = bottleneck at scale**

Spark UI



- **Jobs** and **Stages** - track what ran and how long
- **Shuffle Read/Write** - see how much data moved
- Useful for diagnosing **pipeline bottlenecks**

Reading the query plan with .explain()

```
df_joined.explain(mode="formatted")
```

```
...
+- PhotonBroadcastHashJoin LeftOuter (18)
   :- ...
   +- PhotonShuffleExchangeSource (17)
      +- PhotonShuffleMapStage (16)
         +- PhotonShuffleExchangeSink (15)
            +- LocalTableScan (13)
...
```

- `.explain()` - **inspect the execution plan**
- Nodes 15 & 16 - **no routing Arguments** logged

Broadcast join - the fix

```
# Wrap the small table in F.broadcast()
df_broadcast = df_valid.join(
    F.broadcast(df_dim),
    on="Category",
    how="left"
)

print(f"Broadcast joined rows: {df_broadcast.count():,}")
```

```
Broadcast joined rows: 33,223
```

Before and after - verifying the plan

```
df_broadcast.explain(mode="formatted")
```

```
+- PhotonBroadcastHashJoin LeftOuter (18)
   :- ...
   +- PhotonShuffleExchangeSource (17)
      +- PhotonShuffleMapStage (16)
         +- PhotonShuffleExchangeSink (15)
            +- LocalTableScan (13)
...
(15) PhotonShuffleExchangeSink
Arguments: SinglePartition
(16) PhotonShuffleMapStage
Arguments: EXECUTOR_BROADCAST, [id=#11839]
```

- Dim table has routing instructions
- Large table `df_valid` **never moves**
- One change = **minutes to seconds** at scale

Let's practice!

DATA TRANSFORMATION WITH SPARK SQL IN DATABRICKS